

Tensor.h & Tensor.c

Line-by-Line Explanation for Beginners

The heart of the OnDeviceTraining framework — structs, shapes, and data

1. What is a Tensor?

A **tensor** is the fundamental data container in this framework. Every piece of data — model weights, gradients, activations, input samples, output predictions — is stored in a tensor.

Concept	Real-world analogy	In this project
Scalar (0D)	A single number: 3.14	One weight value
Vector (1D)	A list: [1, 2, 3, 4]	Bias values of one layer
Matrix (2D)	A table with rows and columns	Weights of a Linear layer
3D tensor	A stack of matrices	Conv1d weights: [filters, channels, kernel]

4D+ tensor	A sequence of 3D tensors	Batch of Conv outputs: [batch, filters, channels, time]
------------	--------------------------	---

2. Tensor.h — The Header File

Tensor.h defines the **types** and **function declarations** that the entire framework uses. Any file that works with tensors includes this header.

2.1 Include Guard

<code>#ifndef ODT_TENSOR_H</code>	If ODT_TENSOR_H is not yet defined ...
<code>#define ODT_TENSOR_H</code>	... define it now. Second include is skipped.
<code>... all declarations ..</code>	
<code>#endif // ODT_TENSOR_H</code>	Closes the #ifndef block

2.2 Standard Library Includes

<code>#include <stdbool.h></code>	Provides: bool, true, false
<code>#include <stddef.h></code>	Provides: size_t (unsigned count type)
<code>#include <stdint.h></code>	Provides: uint8_t, int32_t, uint32_t
<code>#include "Quantization.h"</code>	Provides: quantization_t, symQConfig_t ...

2.3 tensorStorageId typedef

<code>typedef void *tensorStorageId;</code>	void* = pointer to anything. An alias for flexibility.
---	--

void * is a generic pointer — it can point to any type. This typedef gives it a meaningful name for use as a storage identifier. It is not used heavily in the current code but exists for future extensibility.

2.4 struct shape (shape_t)

<code>typedef struct shape {</code>	Begin struct definition, aliased as shape_t
<code>size_t numberOfDimensions;</code>	How many axes: 1D=1, 2D=2, 3D=3 ...
<code>size_t *dimensions;</code>	Array of sizes per axis e.g. {64, 128, 9}
<code>size_t *orderOfDimensions;</code>	Axis order — used to transpose without moving data
<code>} shape_t;</code>	End: now shape_t is the type name

Concrete example — shape of a Conv1d input batch:

```

/* Tensor shape: [batch=64, channels=9, time=128] */

numberOfDimensions = 3

dimensions[0] = 64 /* batch size */

dimensions[1] = 9 /* input channels */

dimensions[2] = 128 /* timesteps */

orderOfDimensions = {0, 1, 2} /* default: no transpose */

/* After transposeTensor(t,0,1): orderOfDimensions = {1, 0, 2} */

/* Data bytes stay in place — only the index order changes */

```

Key concept: `dimensions[]` stores the SIZES. `orderOfDimensions[]` stores WHICH axis to read first. Transposing only swaps two entries in `orderOfDimensions` — no bytes are moved. This makes transposition $O(1)$ — instant regardless of tensor size.

2.5 Sparsity Types

<pre> typedef enum { SPARSITY_TYPE_1, SPARSITY_TYPE_2 } sparsityType_t; typedef struct sparsityConfig { } sparsityConfig; typedef struct sparsity { sparsityType_t type; sparsityConfig *config; } sparsity_t; </pre>	Named options for sparsity format Empty for now — placeholder for future config Sparsity descriptor attached to a tensor Which sparsity format is used Pointer to config (NULL if none)
--	--

Note: Sparsity in this project is handled by RigL binary masks (`uint8_t` arrays), not by this `sparsity_t` struct. The `sparsity_t` struct exists as a framework hook for future sparse storage formats (e.g. CSR, COO). For now it is usually set to NULL inside a tensor.

2.6 struct tensor (tensor_t) — THE core type

<pre> typedef struct tensor { uint8_t *data; </pre>	The central data container Raw bytes — the actual numbers
---	---

```
shape_t *shape;
```

Dimensions: how many axes and how large

```
quantization_t*quantization;
```

How to interpret the bytes (float32? int8? bool?)

```
sparsity_t *sparsity;
```

Sparse storage info (usually NULL)

```
} tensor_t;
```

Why is data a uint8_t*?

The tensor stores its numbers as raw bytes. Whether those bytes represent float32, int32, or 4-bit integers depends on the quantization field. `uint8_t *` is the correct type for "a pointer to raw bytes" — it makes no assumptions about what those bytes mean. `DTypes.c` functions are used to read/write the actual values.

```
/* A float32 tensor with 3 values: [1.0, 2.0, 3.0] */
```

```
data bytes: [0x00 0x00 0x80 0x3F] [0x00 0x00 0x00 0x40] [0x00 0x00 0x40 0x40]
```

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

```
1.0f (4 bytes) 2.0f (4 bytes) 3.0f (4 bytes)
```

```
/* All 12 bytes stored as uint8_t[], interpreted as float via quantization */
```

2.7 struct parameter (parameter_t)

```
typedef struct parameter {
```

Groups a weight tensor with its gradient tensor

```
tensor_t *param;
```

The weight values (what the model learned)

```
tensor_t *grad;
```

The gradients (used by the optimizer to update weights)

```
} parameter_t;
```

In neural network training, every learnable weight has a matching gradient. Keeping them together in one `parameter_t` ensures you never accidentally apply gradients from the wrong layer. The optimizer reads `grad` and updates `param`.

3. Tensor.c — The Implementation

Tensor.c implements all the functions declared in Tensor.h. The functions fall into five groups:

Group	Functions	Purpose
Size calculators	calcNumberOfElements*, calcBytes*, calcBits*	Count elements, bytes, bits in a tensor
Bool accessors	tensorBoolGet, tensorBoolSet	Read/write individual bits in a BOOL tensor
Bit manipulation	getBitmask, readByte, writeByte, byteConversion	Low-level bit packing for quantization conversion
Setters	setTensorValues, setShape, setParameterValues	Fill struct fields — the constructors
Copy & Print	copyTensor, printTensor, printShape ...	Debugging and tensor duplication

3.1 Includes in Tensor.c

```
#define SOURCE_FILE "TENSOR"
```

Debug-print label for this file

```
#include <math.h>
```

ceilf() — used for rounding bits to bytes

```
#include <stdio.h>
```

printf() — used in printTensor/printShape

```
#include <stdlib.h>
```

exit() — called on fatal errors

```
#include <string.h>
```

memcpy(), memset() — byte copying and zeroing

```
#include "Common.h"
```

PRINT_ERROR macro

```
#include "DTypes.h"
```

readBytesAsFloat, readBytesAsInt32 ...

```
#include "MinMax.h"
```

min/max helpers (though also defined locally)

```
#include "Quantization.h"
```

quantization_t, all Q type constants

```
#include "Tensor.h"
```

Our own header — must include self

4. Group 1 — Size Calculator Functions

These functions answer three questions: how many elements? how many bytes? how many bits? They are called constantly by the rest of the framework to allocate memory and index into data arrays.

4.1 calcNumberOfElementsByShape

```
size_t calcNumberOfElementsByShape(shape_t *shape)
```

Multiply all dimension sizes together to get the total element count.

```

size_t calcNumberOfElementsByShape(shape_t
*shape) {

    size_t numElem = 1;

    size_t numberOfDimensions =
    shape->numberOfDimensions;

    size_t *dimensions =
    shape->dimensions;

    for (size_t i=0; i<numberOfDimensions; i++) {

        numElem *= dimensions[i];

    }

    return numElem;

}

```

Start with 1 (multiplication identity)

Copy dim count from shape struct

Get pointer to dimensions array

Loop over every axis

Multiply running total by this axis size

e.g. shape [64,9,128] → 64*9*128 = 73728

```

/* Example: Conv1d weight tensor shape [16, 9, 7] */

```

```

calcNumberOfElementsByShape(shape) → 16 * 9 * 7 = 1008 elements

```

4.2 calcNumberOfElementsByTensor & calcNumberOfElementsByParameter

```

size_t calcNumberOfElementsByTensor(tensor_t
*tensor) {

    return
    calcNumberOfElementsByShape(tensor->shape);

}

size_t
calcNumberOfElementsByParameter(parameter_t
*parameter) {

    return calcNumberOfElementsByShape(parameter->
param->shape);

}

```

Delegate to shape version

Use param's shape

Note: Both are thin wrappers. They just reach into the struct to get the shape pointer and call the shape version. This avoids callers needing to know the internal layout.

4.3 calcBytesPerElement

```

size_t calcBytesPerElement(quantization_t *quantization)

```

Return how many bytes each element occupies based on its quantization type.

<pre>size_t calcBytesPerElement(quantization_t *quantization) { switch (quantization->type) { case INT32: return sizeof(int32_t); case FLOAT32: return sizeof(float); case SYM_INT32: return sizeof(int32_t); case SYM: { symQConfig_t *symQC = quantization->qConfig; return (size_t)ceilf((float)symQC->qBits / 8.0f); } case ASYM: { asymQConfig_t *ac = quantization->qConfig; return ceil((float)ac->qBits / (float)8); } case B00L: return 1; default: PRINT_ERROR(...); exit(1); } }</pre>	Branch on quantization type 4 bytes 4 bytes 4 bytes Variable-width symmetric quantization Get config qBits/8 rounded up Variable-width asymmetric quantization qBits/8 rounded up 1 byte per element (packed later) Unknown type: fatal error
--	--

Why `ceilf(qBits / 8.0f)`? If an element uses 5 bits, it still needs 1 full byte (you cannot allocate a fraction of a byte). `ceilf` rounds $5/8 = 0.625$ up to 1.

4.4 calcBitsPerElement

```
size_t calcBitsPerElement(quantization_t *quantization)
```

Return the number of BITS each element needs — same switch, but returns bits not bytes.

```
case INT32: return sizeof(int32_t) * 8; /* 4 * 8 = 32 bits */

case FLOAT32: return sizeof(float) * 8; /* 4 * 8 = 32 bits */

case SYM_INT32: return sizeof(int32_t)*8; /* 32 bits */

case SYM: return symQC->qBits; /* e.g. 8 bits for INT8 */
```

```
case ASYM: return asymQConfig->qBits; /* e.g. 8 bits */

case BOOL: return 1; /* 1 bit per boolean */
```

4.5 calcBitsPerTensor & calcBytesPerTensor

```
size_t calcBitsPerTensor(tensor_t *tensor) {
    size_t bitsPerElement =
        calcBitsPerElement(tensor->quantization);
    size_t numElements =
        calcNumberOfElementsByShape(tensor->shape);
    return bitsPerElement * numElements;
}

size_t calcBytesPerTensor(tensor_t *tensor) {
    size_t bitsPerTensor =
        calcBitsPerTensor(tensor);
    return bitsPerTensor / 8;
}
```

Ask: how many bits is one element?

Ask: how many elements total?

Multiply → total bits

Total bits

Convert bits → bytes

```
/* FLOAT32 tensor shape [64, 9, 128]: */

numElements = 64 * 9 * 128 = 73728

bitsPerElement = 32

bitsPerTensor = 73728 * 32 = 2,359,296 bits

bytesPerTensor = 2,359,296 / 8 = 294,912 bytes = 288 KB
```

4.6 calcNumberOfBytesForData

```
size_t calcNumberOfBytesForData(quantization_t *q, size_t numberOfElements)
```

Like calcBytesPerTensor but takes an explicit element count rather than a tensor.

```
case FLOAT32: return numberOfElements *
    sizeof(float);

case INT32: return numberOfElements *
    sizeof(int32_t);
```

4 bytes each

4 bytes each

<code>case SYM_INT32: return numberOfElements * sizeof(int32_t);</code>	4 bytes each
<code>case SYM:</code>	Variable width
<code>return (calcBitsPerElement(q) * numberOfElements + 7) / 8;</code>	Pack tightly
<code>case ASYM: {</code>	Variable width
<code>size_t bpe = calcBitsPerElement(q);</code>	
<code>return (bpe * numberOfElements + 7) / 8;</code>	Pack tightly
<code>}</code>	
<code>case BOOL: return (numberOfElements + 7) / 8;</code>	8 bools per byte

Key concept: The formula $(bits + 7) / 8$ is "ceiling division" — it rounds up to the next whole byte. For example, 9 bools need 9 bits = $(9+7)/8 = 2$ bytes. Without +7, $9/8=1$ byte which would lose the 9th bool.

5. Group 2 — Bool Tensor Accessors

A BOOL tensor packs 8 boolean values into each byte, one bit per value. These two functions provide get/set access to individual bits.

Why: Why pack bools into bits? A C bool is 1 byte = 8 bits. If you have 8192 booleans, storing each in its own byte costs 8 KB. Bit-packing them into bytes costs only 1 KB. On an MCU with 320 KB total RAM, this difference matters.

5.1 tensorBoolGet — read one bit

```
bool tensorBoolGet(tensor_t const *tensor, size_t flatIndex)
```

Read the boolean at position flatIndex from a bit-packed BOOL tensor.

```
bool tensorBoolGet(tensor_t const *tensor,
size_t flatIndex) {
    if (tensor->quantization->type != B00L) {
        PRINT_ERROR(...); exit(1);
    }
    size_t byteIndex = flatIndex / 8;
    size_t bitIndex = flatIndex % 8;
    return ((tensor->data[byteIndex]
    >> bitIndex) & 1u) != 0;
}
```

Guard: must be BOOL type

Fatal error if wrong type

Which byte holds this bit?

Which bit within that byte?

Read the byte, extract 1 bit

Shift right, mask, compare to 0

How the bit extraction works:

```
/* flatIndex = 11 → we want bit 11 */

byteIndex = 11 / 8 = 1 /* in the second byte */

bitIndex = 11 % 8 = 3 /* bit position 3 within that byte */

data[1] = 0b10101000 = 0xA8

^
```

bit 3

```
data[1] >> 3 = 0b00010101 /* shift bit3 down to position 0 */  
  
result & 1 = 0b00000001 /* mask off everything except bit 0 */  
  
!= 0 → true /* bit 11 is 1 */
```

5.2 tensorBoolSet — write one bit

```
void tensorBoolSet(tensor_t *tensor, size_t flatIndex, bool value)
```

Set or clear the bit at flatIndex in a bit-packed BOOL tensor.

```
void tensorBoolSet(tensor_t *tensor, size_t  
flatIndex, bool value) {  
  
    if (tensor->quantization->type != BOOL) {  
        PRINT_ERROR(...); exit(1);  
    }  
  
    size_t byteIndex = flatIndex / 8;  
    size_t bitIndex = flatIndex % 8;  
  
    if (value) {  
        tensor->data[byteIndex] |=  
            (uint8_t)(1u << bitIndex);  
    } else {  
        tensor->data[byteIndex] &=  
            (uint8_t)~(1u << bitIndex);  
    }  
}
```

Same guard as Get

Which byte

Which bit within byte

Setting to 1:

OR with a 1-bit mask

Mask: only the target bit is 1

Setting to 0:

AND with inverted mask

~mask: all bits 1 except target

Bit operations explained:

```
/* Set bit 3 to 1: */  
  
1u << 3 = 0b00001000 /* mask with only bit3 set */  
  
data |= 0b00001000 /* OR sets bit3, leaves others unchanged */
```

```
/* Set bit 3 to 0: */  
  
1u << 3 = 0b00001000  
  
~(1u << 3) = 0b11110111 /* invert: all 1 except bit3 */  
  
data &= 0b11110111 /* AND clears bit3, leaves others unchanged */
```

6. Group 3 — Bit Manipulation (getBitmask / readByte / writeByte / byteConversion)

These functions handle the low-level bit packing needed to convert tensors between different quantization widths — for example converting a 32-bit float to an 8-bit integer for deployment on the STM32.

6.1 getBitmask — create a bit mask

```
uint32_t getBitmask(uint32_t startbit, uint32_t endbit)
```

Return a byte where bits from startbit to endbit-1 are 1, all others 0.

```
uint32_t getBitmask(uint32_t startbit,
uint32_t endbit) {

    uint32_t endbitInternal = endbit -
    (startbit/8)*8;

    uint32_t startbitInternal = startbit -
    (startbit/8)*8;

    uint32_t counter = 0; uint32_t value = 1;

    for (size_t i = 0; i < 8; i++) {

        if ((i >= startbitInternal) &
            (endbitInternal > i)) {

            counter += value;

        }

        value *= 2;

    }

    return counter;

}
```

Normalise to byte boundary

Normalise to byte boundary

counter accumulates, value = 2^i

Walk all 8 bit positions

Is this bit in the range?

If yes, add 2^i to bitmask

Next power of 2

e.g. bits 2..5 → 0b00111100 = 0x3C

```
getBitmask(2, 6) → bits 2,3,4,5 are set → 0b00111100 = 0x3C
```

```
getBitmask(0, 4) → bits 0,1,2,3 are set → 0b00001111 = 0x0F
```

```
getBitmask(0, 8) → all 8 bits set → 0b11111111 = 0xFF
```

6.2 readByte & writeByte

```
uint8_t readByte(uint8_t data, uint8_t startbit, uint8_t endbit)
```

Extract bits [startbit..endbit-1] from a byte and return them right-aligned.

```
uint8_t readByte(uint8_t data, uint8_t
startbit, uint8_t endbit) {

    uint8_t bitmask = getBitmask(startbit,
endbit);

    uint8_t intermediate = data & bitmask;

    intermediate >>= startbit - (startbit/8)*8;

    return intermediate;

}
```

Build the bit mask

Zero all bits outside the range

Shift right to bit 0

Return right-aligned value

```
uint8_t writeByte(uint8_t existingData, uint8_t data, uint8_t startbit, uint8_t
endbit)
```

Insert bits from data into existingData at positions [startbit..endbit-1].

```
uint8_t writeByte(uint8_t existingData,
uint8_t data,
uint8_t startbit, uint8_t endbit) {

    uint8_t startbitInternal = startbit -
(startbit/8)*8;

    uint8_t endbitInternal = endbit -
(startbit/8)*8;

    uint8_t bitmask = getBitmask(startbitInternal,
endbitInternal);

    data <<= startbitInternal;

    uint8_t intermediate = data & bitmask;

    existingData = intermediate | existingData;

    return existingData;

}
```

Normalise

Normalise

Target mask

Shift data into position

Mask to keep only target bits

OR into existing byte

Return modified byte

6.3 byteConversion — repack bits between different widths

```
void byteConversion(uint8_t *dataIn, size_t dataInBits, uint8_t *dataOut, size_t
dataOutBits, size_t numValues)
```

Convert an array of values stored in dataInBits bits each, to values stored in dataOutBits bits each.

This is the most complex function in Tensor.c. It handles converting tensors between quantization widths — for example repacking 32-bit values into 8-bit slots for INT8 deployment. It processes one value at a time, moving bits chunk by chunk across byte boundaries.

<code>memset(dataOut, 0, (numValues*dataOutBits-1)/8+1);</code>	Zero all output bytes first
<code>size_t dataOutIndex=0, dataInIndex=0;</code>	Byte pointers into each array
<code>int dataOutStartbit=0, dataInStartbit=0;</code>	Bit offsets within current byte
<code>int dataInEndbit=(int)dataInBits;</code>	End of first input value
<code>int dataOutEndbit=(int)dataOutBits;</code>	End of first output slot
<code>for (size_t i=0; i<numValues; i++) {</code>	Loop over each value
<code>while (dataInStartbit<dataInEndbit dataOutStartbit<dataOutEndbit) {</code>	Process all bits of this value
<code>uint8_t data = 0;</code>	
<code>if (dataInStartbit<dataInEndbit)</code>	Read bits from input if available
<code>data = readByte(dataIn[dataInIndex],...);</code>	Extract bits
<code>if (dataOutStartbit<dataOutEndbit)</code>	Write bits to output if space
<code>dataOut[dataOutIndex] = writeByte(...);</code>	Insert bits
<code>/* advance bit pointers ... */</code>	
<code>}</code>	
<code>/* reset bit pointers for next value ... */</code>	
<code>}</code>	

```

/* Example: convert 3 float32 values (32 bits each) to INT8 (8 bits each) */

dataInBits = 32 (each input value occupies 32 bits)

dataOutBits = 8 (each output value occupies 8 bits)

numValues = 3

For each value: copy 8 bits from input into output, discard upper 24 bits

Result: 3 x 32-bit values → 3 x 8-bit values (12 bytes → 3 bytes)

```

7. Group 4 — Setter Functions (Constructors)

These functions fill the fields of a struct. They are the framework's way of constructing tensors, shapes, and parameters — C does not have constructors like C++, so these functions play that role.

7.1 setTensorValues

```
void setTensorValues(tensor_t *tensor, uint8_t *data, shape_t *shape,
                    quantization_t *quantization, sparsity_t *sparsity)
```

Fill all four fields of a tensor_t struct.

```
void setTensorValues(tensor_t *tensor, uint8_t
*data,

shape_t *shape, quantization_t *quantization,
sparsity_t *sparsity) {
    tensor->data = data;

    tensor->shape = shape;

    tensor->quantization = quantization;

    tensor->sparsity = sparsity;
}
```

Pointer to raw bytes

Pointer to shape descriptor

Pointer to Q config

Pointer to sparsity info (often NULL)

Note: This function does NOT allocate memory. The caller must have already allocated the tensor_t struct, the data array, the shape_t, and the quantization_t. This function just wires the pointers together.

7.2 setTensorValuesForConversion

```
void setTensorValuesForConversion(uint8_t *data, quantization_t *q, tensor_t
*originalTensor, tensor_t *outputTensor)
```

Like setTensorValues but copies shape and sparsity from an existing tensor.

```
void setTensorValuesForConversion(uint8_t
*data,

quantization_t *q,

tensor_t *originalTensor,

tensor_t *outputTensor) {

    outputTensor->data = data;
```

New data buffer

New quantization (e.g. INT8 instead of FLOAT32)

Source tensor to copy shape/sparsity from

Destination tensor to fill

New buffer

```

outputTensor->shape = originalTensor->shape;

outputTensor->quantization = q;

outputTensor->sparsity =
originalTensor->sparsity;

}

```

Same shape as original

New quantization

Same sparsity

Used during quantization conversion: the output tensor has the same shape as the input but a different data type and a new byte buffer.

7.3 setShape

```

void setShape(shape_t *shape, size_t *dims,
size_t numberOfDims, size_t *orderOfDims) {
shape->dimensions = dims;
shape->numberOfDimensions = numberOfDims;
shape->orderOfDimensions = orderOfDims;
}

```

Pointer to size array e.g. {64,9,128}

e.g. 3

e.g. {0,1,2}

7.4 setParameterValues

```

void setParameterValues(parameter_t
*parameter,
tensor_t *param, tensor_t *grad) {
parameter->param = param;
parameter->grad = grad;
}

```

Weight tensor pointer

Gradient tensor pointer

7.5 setOrderOfDimsForNewTensor

```

void setOrderOfDimsForNewTensor(
size_t numberOfDimensions,
size_t *orderOfDimensions) {
for (size_t i=0; i<numberOfDimensions; i++)
orderOfDimensions[i] = i;
}

```

Loop every axis

Set axis i order to i (natural order)

Sets the default order: axis 0 first, axis 1 second, etc. For a 3D tensor this gives {0, 1, 2} — no transpose.

7.6 Accessors — getParamFromParameter & getGradFromParameter

```
tensor_t *getParamFromParameter(parameter_t
*parameter) {

    return parameter->param;

}

tensor_t *getGradFromParameter(parameter_t
*parameter) {

    return parameter->grad;

}
```

Return the weight tensor

Return the gradient tensor

Note: These one-line accessors exist so callers do not need to know the internal field names of `parameter_t`. If the struct layout ever changes, only these two functions need updating.

7.7 transposeTensor

```
void transposeTensor(tensor_t *tensor, size_t dim0Index, size_t dim1Index)
```

Swap two axes in `orderOfDimensions` — O(1) transpose with no data movement.

```
void transposeTensor(tensor_t *tensor,
size_t dim0Index, size_t dim1Index) {

    if (tensor->shape->numberOfDimensions < 2)

        return;

    size_t temp =
        tensor->shape->orderOfDimensions[dim0Index];

    tensor->shape->orderOfDimensions[dim0Index] =
        tensor->shape->orderOfDimensions[dim1Index];

    tensor->shape->orderOfDimensions[dim1Index] =
        temp;

}
```

Indices of the two axes to swap

Can't transpose 0D or 1D

Save `dim0`

Set `dim0 = dim1`

Set `dim1 = saved dim0`

```
/* Before: orderOfDimensions = {0, 1, 2} */
```

```
transposeTensor(tensor, 0, 1);
```

```
/* After: orderOfDimensions = {1, 0, 2} */
```

```
/* The data bytes haven't moved — only the index map changed */
```


8. Group 5 — Copy and Print Functions

8.1 printShape

```
void printShape(shape_t *shape) {

    size_t numberOfDims =
        shape->numberOfDimensions;

    printf("NumberOfDims: %lu\n", numberOfDims);

    printf("Dims: \n");

    for (size_t i=0; i<numberOfDims; i++)
        printf("%lu\n", shape->dimensions[i]);

    printf("OrderOfDims: \n");

    for (size_t i=0; i<numberOfDims; i++)
        printf("%lu\n", shape->orderOfDimensions[i]);

}
```

Print how many axes

Loop over dimensions

Print each size

Loop over order

Print each order value

8.2 printTensor

```
void printTensor(tensor_t *t)
```

Print all values in a tensor — format depends on quantization type.

```
void printTensor(tensor_t *t) {
    quantization_t *q = t->quantization;

    printf("TENSOR BEGIN \n");

    size_t numValues =
        calcNumberOfElementsByTensor(t);

    int32_t data[numValues];

    switch (q->type) {
        case FLOAT32: /* readBytesAsFloat per element */
        case INT32: /* readBytesAsInt32 per element */
        case SYM_INT32: /* print scale then int values */
        case ASYM: /* print scale, zeroPoint, then bytes */
    }
```

Get quantization info

Count elements

VLA for reading data

Branch on quantization type

Print as floats

Print as integers

Print scale + ints

Print Q params

```

case BOOL: /* tensorBoolGet per element → 0 or 1 */
    Print as 0/1

}

printf("TENSOR END \n");

}

```

Tip: `printTensor` is a debugging tool. Call it when a layer's output looks wrong and you need to see the actual values. It handles every quantization type automatically — you do not need to know how the data is stored.

8.3 `copyData` / `copyShape` / `copyQuantization` / `copyTensor`

```

void copyData(tensor_t *dest, tensor_t *src) {
    Copy raw bytes

    size_t n =
    calcNumberOfElementsByShape(dest->shape);
    How many elements

    size_t sz =
    calcNumberOfBytesForData(src->quantization,
    n);
    How many bytes

    memcpy(dest->data, src->data, sz);
    Copy the bytes

    if (src->sparsity != NULL)
    If sparsity exists ...

    memcpy(dest->sparsity, src->sparsity,
    sizeof(sparsity_t));
    ... copy it too

}

void copyShape(shape_t *dest, shape_t *src) {
    Copy dimensions

    memcpy(dest->dimensions, src->dimensions,
    Copy sizes array

    src->numberOfDimensions * sizeof(size_t));

    memcpy(dest->orderOfDimensions,
    Copy order array

    src->orderOfDimensions,

    src->numberOfDimensions * sizeof(size_t));

    dest->numberOfDimensions =
    Copy count

    src->numberOfDimensions;

}

void copyTensor(tensor_t *dest, tensor_t *src)
Top-level copy
{
    copyShape(dest->shape, src->shape);
    1. Copy shape

    copyQuantization(dest->quantization,
    2. Copy Q config

    src->quantization);

    copyData(dest, src);
    3. Copy raw bytes
}

```

```
}
```

Warning: copyTensor performs a DEEP copy of the data bytes, but the destination tensor must already have its own allocated buffers for data, shape->dimensions, shape->orderOfDimensions, and quantization->qConfig. It does NOT allocate memory — it only copies values into existing memory.

9. Summary — How It All Fits Together

Here is how the structs nest inside each other and how the functions work on them:

[illegible]

Function category	Functions	Used by
Count elements	calcNumberOfElements*	Memory allocation, loop bounds
Count bytes/bits	calcBytes*, calcBits*	Serialisation, buffer sizing
Bool bit access	tensorBoolGet/Set	Packed boolean tensor R/W
Bit packing	getBitmask, readByte, writeByte, byteConversion	Quantization conversion
Construct	setTensorValues, setShape, setParameterValues	TensorApi.c, model init
Navigate	getParamFromParameter, getGradFromParameter	Copier, RigiL
Transform	transposeTensor	Conv layers (reorder axes)
Debug	printTensor, printShape	Development only
Duplicate	copyTensor, copyShape, copyData, copyQuantization	Framework class cloning